Chapter 5

DevSecOps Technology

Introduction

DevSecOps technology enables and underpins the core principles of DevOps, but it is important to reiterate that DevSecOps and DevOps are not just about technology. DevSecOps is primarily about culture and the DevOps principles. With this in mind, it is important to clarify just what constitutes a DevSecOps tool. What defines DevSecOps technology? The reality is that there is no such thing as a "DevSecOps" tool, but, rather, a set of tools that enable DevSecOps. While this may seem like a subtle differentiation, it is an important one because how a tool is used impacts the outcome. If you think of DevSecOps as a culture of collaboration, then you can consider a set of tools that enable collaboration. More generally, you can think of DevSecOps tools as the set of security tools that enables the core principles of DevOps.

The reality is, there is no such thing as a "DevSecOps" tool, but, rather, a set of tools that enable DevSecOps. . .. If you think of DevSecOps as a culture of collaboration, then you can consider a set of tools that enables collaboration.

Marketing messages make this point more confusing, as many modern security tools now purport to be a "DevSecOps tool." It is important to consider what a DevSecOps tool actually is to separate reality from the marketing messages. Tools like *extended detection and response* (*XDR)*, which monitor the environment, focus on core principles such as observability and transparency. However, if these tools are not shared with others who are responsible for these systems, they can actually be contradictory to DevOps principles. Tools are, ultimately, just a means to an end and can be used in many different ways. So, when understanding if something is a "DevSecOps tool," you must

look at tools that enable the core principles of DevOps. You must understand how to use technology to enable collaboration, rapid feedback, continuous learning, and small batch delivery.

You must look for tools that enable DevSecOps principles such as these:

- **Collaboration**—As collaboration is at the core of DevSecOps, you must look for tools that enable better collaboration across teams. Chat tools such as Slack or Teams promote the free flow of information required for collaboration in global hybrid team environments. In addition, tools such as PagerDuty can help coordinate and communicate between teams when issues arise.
- **Flow of value from left to right**—CI/CD tools such as Jenkins and CircleCI are the key to Gene Kim's First Way of DevOps, allowing for the smooth and continuous flow of value to the customer. In addition, automation in its many forms helps eliminate daily toil, thereby allowing engineers to focus on building great products and services for the customers. This automation can take the form of security orchestration, automation, and response (SOAR) tools or custom scripts. As long as the focus is on reducing manual work, they help support flow of value to the customers.
- **Empowerment**—By giving engineers more control over the application life cycle from development to production support, you empower them, which drives better business results. Abstraction layers such as cloud providers and automated security testing allow engineers to do the right thing without requiring hand-offs or dependence on other teams.
- **Fast feedback and continuous learning**—Small batch delivery enabled by CI/CD tools enables rapid feedback from the customers, thus allowing engineers to rapidly see the impact their changes are having for their customers. In addition, shared telemetry from monitoring and observability tools can provide critical visibility to the impact of changes.

- **Shift Left**—Using test-driven development and inserting security testing in the pipeline can increase the quality of your service and give you more confidence to release faster and more frequently.

The Fallacy of a "DevSecOps Tool"

No tool is inherently a "DevSecOps tool." This is because alignment with DevOps principles is dependent on how a tool is used. I have seen tools designed for, and marketed as, DevSecOps used in ways that are fundamentally at odds with the core DevOps principles.

During my time running a global DevOps consulting firm, we worked with one team that had three different chat platforms. The development team used Slack, the security team used Teams, and the business teams used HipChat. This use of these tools actually reenforced silos and discouraged communication between the teams. In this way, chat tools, arguably one of the tools that can clearly enhance collaboration, was being used in a way that actually prevented collaboration across teams. When considering what a "DevSecOps tool" is, it is important to look not just at the tool's function, but also at how it is being used.

DevSecOps Continuous Integration and Continuous Deployment

Continuous integration and continuous deployment (CI/CD) is a cornerstone of DevOps principles and practices, as it enables small batch delivery by providing short feedback loops and strengthening input from the customer. The CI/CD process includes all the steps to take changes to code from the developer and get them built and deployed to the customer. This includes the following steps:

1. Pushing the code from the source code repository
2. Integrating it into the rest of the application

3. Building a complete application
4. Testing the application
5. Deploying the application to production all through automation

This is in stark contrast to older methods, where developers would manually build their code in isolation and then send it to testers to examine, who would log and report any errors they encountered.

CI/CD is the process for rapid building and deploying of applications. Continuous integration allows developers to integrate their code with the code of other developers who are working on the same application or service at any time. Continuous deployment takes this code and builds it into a deployable state so that it can be pushed to production at any time. It should be noted that continuous deployment does not mean that applications are always deployed to the customer, only that the application is always in a deployable state. While delivering the product to customers quickly is important, constant changes to the application may cause a suboptimal experience for the end users. Timing is crucial in determining when certain changes are launched. Therefore, continuous deployment pipelines may simply build the production-ready package to be available for deployment when scheduled or may deploy every change depending on the business requirements.

CI/CD provides amazing improvements in direct feedback to the developers. By getting small incremental changes to customers quickly, engineers get rapid feedback on the impact their changes are having for their customers. The deployment pipeline also provides tremendous opportunities for security. By incorporating security tools in the deployment process, you can ensure that code is secure before it ever gets to a production environment.

Figure 5.1 shows a basic CI/CD diagram. At each stage of the deployment pipeline, different tools can be used to ensure the security of the release. These tools, as well as a selection of the top vendors in each area, are listed. While this is not a comprehensive picture of all available tools or providers, it should provide some idea about the

variety of vendors available to you as you build out your pipeline. Note that some tools are available in multiple stages because they can be used at multiple points throughout the CI/CD pipeline. The following sections go into more detail about the stages of the deployment pipeline and the tools that can be used at each stage.

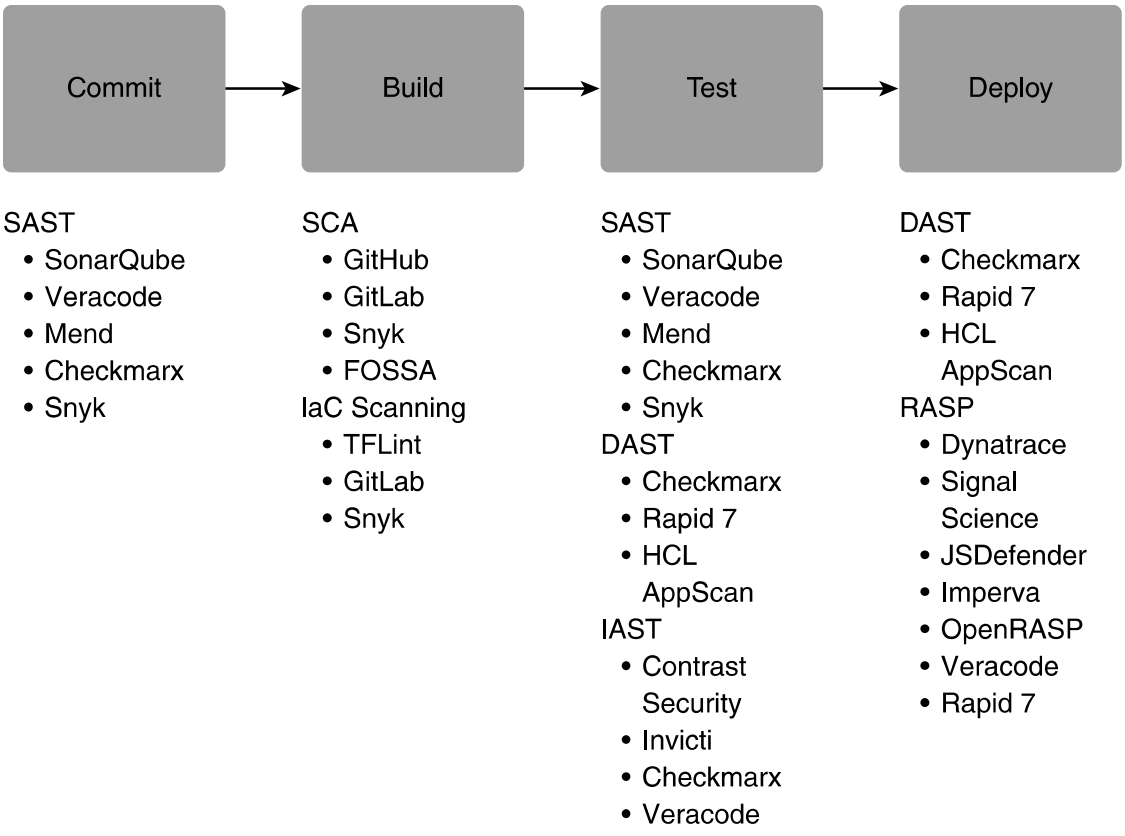


Figure 5.1 The DevSecOps pipeline integrates security tools directly into the process of building and deploying applications and services.

The Commit Stage

Commit is generally the first step in the DevOps CI/CD pipeline. Developers commit their code to a source code repository to provide a centralized source for storing code and tracking versioning accurately. To appropriately manage the code, all source code should be checked into a shared repository. There are many source code repositories including GitHub, Bitbucket, SourceForge, and GitLab. Cloud providers have their own source code repositories, such as AWS CodeCommit and Azure Repos.

The commit stage is a great early opportunity to verify the security of the code. Static code analysis tools such as static application security testing (SAST) can insert security at the first step in the CI/CD pipeline. SAST can analyze the code committed to determine if there are any potential security weaknesses, such as cross-site scripting, SQL injections, and buffer overflow. Popular SAST tools include SonarQube, Veracode, Mend, Checkmarx CxSAST, and Snyk. SAST provides *white-box analysis*, looking directly at the code and offering considerable protection because of the full visibility into the inner workings of the system.

SAST tools can be built into the deployment process and run every time new code is checked in. The CI/CD orchestration tool can then be configured to reject or flag any potential issues. By detecting and rejecting these issues early in the CI/CD process, developers can ensure that security issues are never introduced to the codebase. In addition, by providing immediate feedback to the developer, the cost to remediate these issues is lower. Identifying issues earlier in the software development life cycle is a great example of the concept of Shift Left.

The Build Stage

The build stage of the CI/CD pipeline takes newly committed code and integrates it with existing code to build the system or service. Building will also pull in any dependencies or libraries that need to be included in the final product. The build stage usually produces a completed package that is ready to be tested and deployed.

A standard build process may include the following steps:

1. Compilation of all code that must be built to develop executable programs
2. *Linting*, which checks the code for programmatic and stylistic errors
3. Artifact generation, where the final package is prepared for deployment to production[1]

[1] Fernandez, Tomas. "Design an Effective Build Stage for Continuous Integration." Semaphore. December 7, 2022. `https://semaphoreci.com/blog/build-stage`.

The build phase should be used to incorporate any security standard tools and libraries. During the build phase, you can incorporate code for endpoint protection, dynamic application testing, and logging and tagging standards. You should use this opportunity to ensure that all your standard security tools are built into the product. This should include installation of endpoint detection and response (EDR) tools as well as dynamic application security testing (DAST) tools. Although these are runtime tools, you can use the build phase to ensure they are built into your solution. This is also a good time to verify that the tagging standards for your organization are adhered to, both for the application and for infrastructure deployments.

During the build phase, you can also use source composition analysis (SCA) to check for vulnerabilities in any dependencies or libraries. Third-party vulnerabilities are increasingly becoming a major area of risk for companies, as seen in the Apache Log4J breach. SCA looks at all source code to determine potential known vulnerabilities in linked libraries and open-source code references. SCA can also be used to identify any potential license violations that could cost companies a significant amount of money. Top SCA tools include GitHub, GitLab, Snyk, and FOSSA. By scanning at the build phase, you can ensure that the code that's submitted is secure and that external dependencies, such as open-source libraries, are secure as well.

The build phase is also a good point to scan any IaC for vulnerabilities related to components of your infrastructure. IaC scanning works in a similar manner to SAST tools, except that they focus on infrastructure-related vulnerabilities. Key tools in this category include TFLint, GitLab, and Snyk. By leveraging IaC scanning, you can ensure that your application and your infrastructure is secure.

The Test Stage

Once the build process is complete, automated testing of the compiled system can begin. The key for DevSecOps practices is to make security testing part of the process. This testing will be *black-box testing*, looking for ways to compromise the build system as a compiled whole. DAST can be deployed at this stage. DAST provides an outside-in evaluation of the applications and services. It helps identify vulnerabilities by

simulating attacks in a way similar to how an external party would try to breach the application or service.

DAST tools are run against the compiled product. As part of the deployment process the application or service should be deployed to a staging environment where this sort of testing can be run. DAST tools search for vulnerabilities in a running application, such as cross-site scripting, external XML entities (XXE), cross-site request forgery (CSRF), or SQL injections that your SAST tool may not have caught. While static code analysis may identify some of these issues, it is important to run these tests against a running application. Because DAST tools are designed to be run against a dynamic environment, they can identify issues that your SAST tool may not.[2] It should be noted that while DAST tools are quite useful in identifying potential issues, they are inherently noisy and tend to produce a lot of false positives.

The Microfocus blog says "a DAST scanner can be thought of like a security guard. However, rather than just making sure the doors and windows are locked, this guard goes a step further by attempting to physically break into the building. The guard might try to pick the locks on the doors or break windows. After finishing this examination, the guard could report back to the building manager and provide an explanation of how he was able to break into the building." To extend this metaphor, SAST examines all the components of the lock to ensure the integrity of each one. DAST attempts to pick the fully assembled lock and your CI/CD pipeline coordinates all phases of the process, assembles and installs the lock, and coordinates the testing.

Interactive application security testing (IAST) provides another useful tool during the testing phase. IAST solutions instrument applications by deploying agents and sensors in running applications to analyze how the application performs during runtime, which can be simulated by testing. Unlike SAST and DAST, IAST is integrated into your codebase and runs when your code runs. It should be noted that because IAST runs only when your code runs, it will not test all code

[2] "What Is Dynamic Application Security Testing (DAST)?" OpenText. March 1, 2023. www.microfocus.com/en-us/what-is/dast.

but only those code paths that are executed during runtime. IAST provides instrumentation of the application code designed to identify and alert potential attacks and provides a useful tool in detecting security issues during functional and performance testing.

The Deploy Stage

The deployment phase makes the code available to users of the application or service. This is the point at which you transition from looking at the build time security vulnerabilities to identifying runtime vulnerabilities. Once an application or service has been pushed to production, the feature is available but so are any vulnerabilities or security flaws that still exist, which is why it is so critical to find and address any issues prior to this stage. In addition, new vulnerabilities may be discovered in already deployed code, so it is important to continuously scan the production environment for vulnerability.

The deployment stage also represents the point at which security logging, telemetry, and IAST happen. This is the point at which you can begin to see how the application performs under real-world scenarios and, at times, real-world attack scenarios. IAST runs in real time during application execution. In addition, runtime application self-protection (RASP) can detect intrusion attempts and respond to remediate those threats. Real-time logging and monitoring provide valuable insights into how the system is operating in production, thus helping to alert you to intrusion attempts and other malicious activities.

IDE Integration

Many companies are now integrating security testing directly into the tools that developers are using to develop their code, their integrated development environment (IDE). This extends the Shift Left paradigm, moving the security testing earlier in the development process and providing faster feedback to the developer. By implementing security testing directly into the IDE, developers do not need to wait to commit their code before they receive feedback.

The IDE is a great place to do static code analysis and check dependencies such as libraries using SAST and SCA. Tools such as Snyk and Contrast now offer the ability to integrate directly into the IDE, providing immediate feedback as the developer is working. This is akin to a word processor that highlights misspelled words as opposed to submitting a completed paper to someone else and then waiting for them to edit and return a marked-up version. Clearly, by providing direct and immediate feedback, the author of the code is empowered to take security concerns into their own hands without reliance on others, thus reducing the need for unnecessary handoffs.

While integration of security tools into the IDE provides a significant opportunity to shorten feedback loops, it will not provide testing of the fully built application nor will it catch runtime dependencies or even issues that occur when code is integrated with other code components. Therefore, it should not be seen as a substitute for IAST or DAST.

INFRASTRUCTURE AS CODE

IaC is another DevOps-related technology with significant opportunities for cybersecurity. IaC allows developers to describe the systems on which an application will run using code enabling the automated deployment not just of the application, but also the infrastructure on which it runs. Tools such as Terraform, CloudFormation, and Azure Resource Manager (ARM) allow engineers to describe the exact infrastructure as code.

IaC makes it easy to ensure consistency between development, testing, and production environments. It also provides the ability to easily re-create the entire environment from the ground up in disaster recovery scenarios. Because the code is checked in and managed by a source code repository, it also allows for changes to be easily rolled back if they are problematic. IaC ensures consistency, repeatability, traceability, and resilience.

The following Terraform code from HashiCorp's tutorial provides a basic example of code used to describe and deploy infrastructure:

```
terraform {
   required_providers {
      aws = {
      source  = "hashicorp/aws"
     version = "~> 4.16"
   }
 }
   required_version = ">= 1.2.0"
}
   provider "aws" {

 region  = "us-west-2"

}
resource "aws_instance" "app_server" {

ami           = "ami-830c94e3"

instance_type = "t2.micro"
tags = {
      Name = "ExampleAppServerInstance"
 }
}
```
[3]

This code will automatically create a new server instance in the AWS US West region. This code is used to specify where the server will be deployed, what type of system it will be (`t2.micro`), and the name of the system (`ExampleAppServerInstance`).

While this simple example configures only a single application server, languages such as Terraform are immensely powerful and can be used to deploy highly complex environments.

Once this is executed, the infrastructure will be available for deploying the application. Additional tools such as Ansible can then be

[3] "Build Infrastructure." HashiCorp. Accessed April 18, 2023. `https://learn.hashicorp.com/tutorials/terraform/aws-build`.

used to deploy detailed system configuration. IaC can be integrated into the CI/CD pipeline so that it not only deploys the latest version of the code but also deploys the infrastructure as well as deploying everything needed for a fully functional system.

As with other Shift Left practices, IaC poses risks and opportunities. As IaC relies on templates for the deployment of infrastructure, an insecure template can easily replicate vulnerabilities across many environments. By the same light, that the template conforms to security standards, those standards will be present in all infrastructure that leverage those templates.

You can also use some of the same tools discussed for application security within your deployment pipeline to verify the security and compliance for infrastructure as code. SAST tools such as XenonStack and Snyk provide code analysis tools for your IaC code. And, because you are talking about code, you can build these checks into your deployment pipeline to ensure that it is secure.

One of the other security benefits that IaC offers is traceability. Because your infrastructure is code, you can treat it as such by checking it into a code repository and submitting any changes. Rather than allowing engineers to make changes directly via your cloud provider or hardware interfaces, all infrastructure is checked in, and any changes to it can be tracked. Like application code, all changes should be required to be reviewed and approved, another safeguard often not offered when hardware is manually configured.

SECRETS MANAGEMENT

To securely build a successful CI/CD pipeline, it is important to carefully manage credentials including usernames and passwords as well as private keys. Whether these be credentials used to connect the application to a data source or the credentials needed to automatically deploy infrastructure, these secrets must be carefully managed. These secrets cannot be stored in the source code repository, where they are easily discoverable by potential threat actors even if the source code is protected. In addition, similar to shared passwords, storing secrets in the source code repositories can lead to a lack of knowledge of who has

access to which systems and easily give people access that they should not have. Even worse, these secrets are often inadvertently exposed on the open Internet for anyone to access.

Secrets management provides a set of tools to securely store any credentials and other sensitive information that can then be accessed by applications with the appropriate credentials. These secrets include passwords, API keys or credentials, and tokens or certificates. Common secrets management tools include Vault AWS Secrets Manager and Azure Key Vault. These tools enable many developers to leverage the same set of secrets without even knowing them or having access to them.

The following IaC example from HashiCorp uses their secrets manager, called Vault, to retrieve temporary credentials that are then used to create compute resources (EC2 instances) in AWS:

```
variable "name" { default = "dynamic-aws-creds-operator" }
variable "region" { default = "us-east-1" }
variable "path" { default = "../vault-admin-workspace/
terraform.tfstate" }
variable "ttl" { default = "1" }

terraform {
  backend "local" {
    path = "terraform.tfstate"
  }
}

data "terraform_remote_state" "admin" {
  backend = "local"

  config = {
    path = var.path
  }
}

data "vault_aws_access_credentials" "creds" {
  backend = data.terraform_remote_state.admin.outputs.
backend
  role    = data.terraform_remote_state.admin.outputs.role
```

```
}

provider "aws" {
  region     = var.region
  access_key = data.vault_aws_access_credentials.creds
.access_key
  secret_key = data.vault_aws_access_credentials.creds
.secret_key
}

data "aws_ami" "ubuntu" {
  most_recent = true

  filter {
    name   = "name"
    values = ["ubuntu/images/hvm-ssd/ubuntu-trusty-14.04-
amd64-server-*"]
  }

  filter {
    name   = "virtualization-type"
    values = ["hvm"]
  }

  owners = ["099720109477"] # Canonical
}

# Create AWS EC2 Instance
resource "aws_instance" "main" {
  ami           = data.aws_ami.ubuntu.id
  instance_type = "t2.nano"

  tags = {
    Name  = var.name
    TTL   = var.ttl
    owner = "${var.name}-guide"
  }
}[4]
```

[4] Nguyen, Tu. "Build Infrastructure." GitHub. HashiCorp, July 13, 2020. https://github.com/hashicorp/learn-terraform-inject-secrets-aws-vault/blob/main/operator-workspace/main.tf#L20.

Privileged Access Management

Privileged access management (PAM) is a cybersecurity mechanism for protecting access to resources from databases to infrastructure to codebases. PAM provides monitoring and detection of unauthorized access as well as providing mechanisms for safely, allowing the right level of access to the appropriate users. There are several tools and combinations of tools that can aid in PAM. One of the benefits of PAM solutions is that they can allow temporary access just in time to a select user, thereby providing the minimal access required without preventing access that is needed. If an engineer needs access to a system to troubleshoot an incident, they can be granted temporary access to that system or to the set of impacted systems. Instead of providing blanket access for an indefinite period of time to a large group of users—as with older identity approaches—PAM solutions can allow temporary access. This aligns very well with the Zero Trust approaches to identity discussed in Chapter 2, "The Evolution of Cybersecurity (from Perimeter to Zero Trust").

A DevSecOps approach to PAM requires a new way of thinking about access management. While you still need to ensure people have the least privileges needed to do their job, you need to think carefully about enablement and empowerment. One of the best ways to disempower someone is to not give them the ability to execute their job or to require them to ask permission to do the basic tasks needed to do their job. It is important to stop thinking of security as a gate, and instead think of it as guardrails. PAM can be a powerful tool to allow people to have access to the systems they need in emergency situations, so it is very important that this access be quickly implemented. Instead of thinking about how to stop people from doing stupid things, start thinking about how to enable them to do their jobs in a safe way. While this is a subtle difference, it is critical when considering the approach to identity and access management.

In thinking about the implementation of PAM, it is important to consider the access that people need to do their jobs. If you are asking developers to be responsible for runtime security of the platforms and services they build, they need access to the security telemetry for those systems. If you are asking teams to be responsible for responding to

incidents, you should empower them to easily push updates to resolve the issues. This is not to say that you simply allow everyone to have access to everything all the time. Quite the opposite. PAM gives you the ability to provide granular control for specified time windows. It also allows you to monitor more closely for dynamic indicators of compromised accounts. By leveraging all the capabilities of PAM, you can ensure that everyone can do their jobs seamlessly, without compromising security.

RUNTIME APPLICATION SELF-PROTECTION

Runtime application self-protection (RASP) tools are another powerful bullet in your DevSecOps arsenal for managing operational activities. RASP uses information about an application's internal state to identify potential threats at runtime, which may not have been identified earlier in the process. When RASP tools detect potential threats, they can automatically take action to address the threat. RASP leverages the internal application logic to detect potential attacks. Based on the attack, these tools can take actions—such as isolating, blocking, or alerting—to ensure that the attack is prevented.

RASP acts like an internal sensor similar to an internal motion sensor that might detect an intruder and sound an alert or automatically lock doors when intrusion is detected. Because RASP tools are built into the application, they have insight into the internal application execution, thus providing deeper visibility into what might constitute a threat. In addition, they can take an active role in defending against the attack. Because of the automated nature of RASP tools, they help eliminate toil and become an important part of the DevSecOps toolset.

MONITORING AND OBSERVABILITY

Monitoring and *observability* technologies are core to DevSecOps because these extend beyond Agile deployment into the operational aspect of a system. Monitoring and observability tools provide insight into potential system issues in near real time, allowing developers to

detect security attacks before they impact their services. These tools also help them understand the scope of security incidents, identify the cause, and implement remediation as quickly as possible.

Monitoring

Monitoring is the process of observing how a system behaves. Monitoring relies on the *telemetry* of a system, where telemetry is defined as the data that is recorded and transmitted about how a system behaves. At a base level, this telemetry can include system-level metrics such as CPU levels. It can also include data from monitoring tools, such as endpoint detection and response (EDR), which sit on system endpoints and issue alerts when they detect potential security issues. In line with the concepts of Defense in Depth, security monitoring can and should sit at every layer of your application, from the database to the user experience. Monitoring should also include monitoring of your logs for patterns, which may be indicative of a security issue. Something as simple as a large spike in log messages about failed authorization may be indicative of an intrusion attempt.

One of the toughest challenges that many midsize to large enterprises face today is the massive growth of monitoring data and the proliferation of monitoring tools. With the increasing size and complexity of systems has come an increasing number of monitoring points and data about those points. In addition, there seems to be a never-ending flood of tools to help you monitor the security of your systems. However, this proliferation of tools and data points can actually be detrimental to your ability to rapidly identify real security threats and issues.

Monitoring Tool Proliferation

At one company I worked, we had more than 50 monitoring tools, from database to system to real user monitoring! At each layer, there were multiple tools duplicating the same type of monitoring. Not only that, but even where the same tools were

(*continues*)

(*continued*)

used, the implementation was different so that they could not be used in the same way. In one data center, alerts were arranged by application; in another they were arranged by rack number. On one tool a red alert meant something completely different than a red alert on another system.

Because of the number of tools and alerts, it was practically impossible to identify when an actual problem was occurring. Even with a modern *security operations center (SOC)* with a full wall display there was nowhere enough room to begin to look at all of the monitoring dashboards and information. Postmortems would often reveal that an alert had fired but that no one had seen it.

We undertook a massive effort to redefine our monitoring stack, looking for duplication and removing tools wherever possible. We rationalized the tools from 50 down to just 12. We also worked diligently to define standards so that monitoring tools were used in a uniform manner across all of the systems. The final step in the process was to identify and reduce alert noise. We created a rule that any alert must be acted upon, even if that action was to turn off the alert if it was not useful. With this effort, the team significantly improved its ability to detect and prevent potential threats and to drive resolutions quickly when problems did arise.

Because of the continued increasing size and complexity of systems, monitoring is rarely sufficient and quickly becomes unmanageable without additional tools to correlate and present the data. In some cases, telemetry in isolation can be of limited use. For example, CPU utilization data in isolation offers very little value as it may or may not have a bearing on system performance or customer experience. What does it tell you if the CPU is at 100 percent utilization? Is this a good thing or a bad thing? In some systems you may very well want to use the maximum available processing power, so higher CPU utilization

may actually be an indicator of optimum performance. In other systems, it may be an indicator of a *Distributed-Denial-of-Service (DDoS)* attack. What matters most is the user experience, so it is critical to collaborate with users early in the development process to carefully define and monitor performance against these expectations. However, especially in cybersecurity, user experience alone is not enough because there may be no impact to user experience in the case of a breach or attempted breach.

Because of the proliferation of data sources, it is important to leverage tools to provide correlation and presentation of the data to rapidly identify and address potential security issues. Correlation takes the telemetry from many data sources and brings it together, helping to identify duplicates and potentially related events from separate systems. For example, CPU spiking alone may not be indicative of a problem, but CPU spiking in conjunction with alerts from network detection and response (NDR) system and increased response times from synthetic user monitoring may be indicative of a DDoS attack. *Security information and event management (SIEM)* tools such a Splunk and Datadog attempt to take all of the various data sources, correlate them, and present them in a way that is easily digestible and actionable by humans. Figure 5.2 provides a basic representation of a correlation tool, which takes data from many sources to provide a useful representation of the data via a presentation layer.

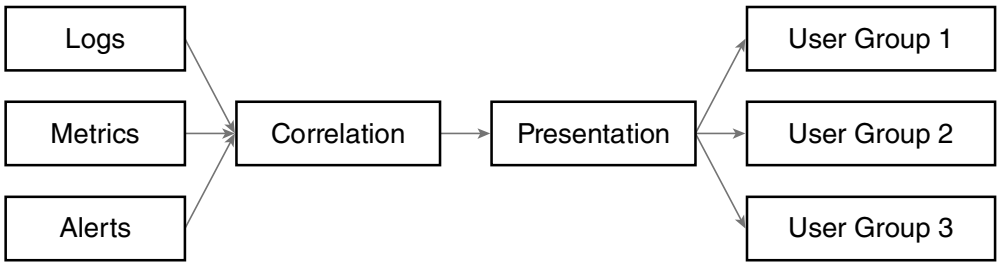


Figure 5.2 Correlating events across all logs, metrics, and alerts provides a variety of views for the many users of monitoring data.

Observability

Observability takes the next step in managing the operations of complex systems, as it focuses on answering novel questions about the current status of the system. *Observability* is a measure of how well

internal states of a system can be inferred from knowledge of its external outputs. Monitoring and observability are distinct concepts that depend on each other. Monitoring is an action you perform to increase the observability of your system. Observability, on the other hand, is a property of that system, like stability or maintainability. One important difference between monitoring and observability is that monitoring requires you to know, in advance, what you want to know about. Because of the complexity of systems today, it is often impossible to predict what you will need to know about. Observability tools, on the other hand, allow you to ask new questions about the status of a system.

The concept of observability actually dates back to the 1960s when the concept was introduced by the Hungarian-American engineer Rudolf E. Kálmán, in the field of control theory, the mathematical study of systems, such as industrial processes. Kálmán states that a system was observable if its internal state could be inferred from the external outputs. In recent years, this same concept has been extended to technical applications and services.

The three pillars of observability are metrics, logs, and traces. These pillars can be understood as follows:

- **Metrics**—Metrics represent key-value pairs of information about a system. Metrics might include information such as the number of transactions per second or CPU utilization levels. This is often time-series data.
- **Logs**—Logs are representations of system status or events that are stored in a file.
- **Traces**—Traces represent the flow of activity through a system, with insight into each step a single transaction might take.

Each of these pillars has unique benefits as well as shortcomings. When the three pillars of observability are brought together, you can gain a wholistic picture of a system's performance. New Relic expanded on the idea with the MELT model, which includes metrics, events, logs, and traces, where events represent an action happening at a precise moment, such as when a system exceeds a specified threshold.

Data Silos

While DevSecOps often focuses on eliminating organizational and process silos, there are frequently data silos that underpin these organizational silos. In the field of security, the isolation of data can, at times, be necessary for confidentiality or privacy reasons. However, more often than not, these data silos are unnecessary and can impede the flow of information across the organization. You see these data silos when development teams have the data they are looking at, such as defects, and operations, has a separate set of data including things like log files or real user monitoring, and security has another set of data, such as vulnerability data and intrusion detection alerts. In addition, the business users often have a completely separate set of data that they are looking at, such as revenue or user conversion rates.

Data Silos' Impact on Incident Resolution

I have all too often been on incident bridges and heard these data silos play out in real time with the security team looking at one set of metrics, the operations team looking at another, and the infrastructure teams looking at a completely separate set of information, with no team having visibility into the other team's data.

The network team will report, "We're not seeing any problem on our side."

The database team will report, "No issues on the database."

The development team will report, "Application is not reporting any errors."

But the incident managers, and, frequently, the customers, know there is a problem. However, the lack of transparency impairs collaboration, leading to lack of trust and delays in resolution.

The *DevOps Handbook* speaks about exactly these issues, stating, "for decades we have ended up with silos of information, where Development only creates logging events that are interesting to developers, and Operations only monitors whether the environments are up or down. As a result, when inopportune events occur, no one can determine why the entire system is not operating as designed or which specific component is failing, impeding our ability to bring our system back to a working state."[5] Unless these data silos are broken down, they will serve only to emphasize existing organizational silos. However, a proper approach and architecture for your monitoring data can help.

Fortunately, you can take steps to help eliminate data silos. One of the first steps is to fix the data coming in. One key step here is to look at the various inputs and reduce duplicate systems wherever possible. If there are two systems used for the same purpose, it is beneficial to eliminate one. This often happens in legacy systems, where previous functionality and data were used differently under different management or business features and not properly eliminated, modified, or turned off. While flexibility and autonomy are important in DevSecOps, this is one area where it can be beneficial to consolidate and align across the organization. This is not to say you should have one tool to do all the monitoring. At the same time, having five different tools to monitor the CPU is clearly also not ideal. While it is good to have generalizable tools that cover multiple types of monitoring, it is also important to identify key point solutions. You may, for example, require a specific tool for vulnerability management in your cloud environment separate from your data center environment. The number of tools and the focus of the tools will differ based on the size and maturity of an organization. What is important is that you reduce redundancy in your telemetry wherever possible.

Wherever possible, it is also important to drive consistency across your data sources. For tools that are part of the distributed tracing chain, you should make sure that the trace ID is included in metrics, logs, and alerts. In addition, key values such as application ID must be included in your data sources. If one tool refers to an application by ID

[5] Kim, Gene, Jez Humble, Patrick Debois, and John Willis. 2021. The DevOps Handbook: How to Create World-Class Agility, Reliability, & Security in Technology Organizations. 1st ed. Portland: IT Revolution Press.

and another uses another name, it will rapidly become difficult if not impossible to correlate the data across the disparate tools throughout the environment.

Once source data has been appropriately addressed, the data can be brought into a unified data store. The data store can be centralized or distributed depending on your size and the distribution of the application landscape. What is important is that you can leverage key data across all sources.

This same approach should be used for security. All too often, security data is siloed away in the name of "security," when the real intent is job security. Data is not shared as a way for individuals to maintain control and prevent others from easily replacing them. But you must share data carefully, as not all users should have access to all data. Transparency is good, but complete transparency could expose sensitive information about your vulnerabilities to malicious actors. Trust is also important, but insider threat is a reality so you need to ensure that sensitive data is exposed only to the right people. This can be accomplished by managing access to the data through access controls. When data is appropriately categorized, it can be segregated so that the appropriate people have access to the data they need. In this way, you can share data effectively across the organization to drive better collaboration without exposing vital security information.

Event Management with SIEM and SOAR

Because of the quantity of information that can be produced during production, SIEM tools can be used to collect all of the data and help security operations teams and security engineers use the data. As mentioned earlier, SIEM tools collect data from all of the sources of security-related information including logs, metrics, traces, and events to provide unified insight and management of this data. These tools provide real-time analysis of this data, helping make sense of the vast amount of information from monitoring and logging systems and helping identify potential attacks. SIEM tools can also assist in incident troubleshooting and response by identifying the cause of the incident and assisting with automation.

SIEM tools can detect a security incident and all of the alerts related to the incident and present it in a way that is easily actionable. For example, during a DoS-type attack, there may be hundreds or even thousands of alerts as systems try to respond to the flood of requests. SIEM tools can correlate these alerts and suppress duplicates, making it easier to diagnose the problem and drive a quicker resolution.

SOAR tools help automate many of these sorts of activities. SOAR refers to a collection of tools that help collect data on vulnerabilities and security events and automate the responses. These tools can be built to automatically respond to known threat types. If the preventative actions for certain types of attacks are known, these responses can be automated using SOAR. With the focus on increasing automation to reduce toil, SOAR tools can be an important part of the DevSecOps toolset.

CONCLUSION

While DevSecOps is not about technology, tools, CI/CD pipelines, and automation are essential to enabling the principles of DevOps within a security context. Build a culture of collaboration—tools alone do not make you "DevOps."

Build a culture of collaboration—tools alone do not make you "DevOps."

DevOps focuses on the CI/CD pipeline because it enables small batch delivery, which, in turn, allows for the flow of value to the customer, shorter feedback loops, and experimentation, *all* of the Three Ways of DevOps. By integrating security into the CI/CD pipeline, you can extend the benefits of DevOps to security, thus enabling the DevSecOps culture.

When you use privileged access management and secrets management correctly, you empower engineers with the access they need to do their jobs quickly and efficiently, ensuring they aren't blockers. In doing so, you enable a state of flow in the creative process of developing great technology, which is critical to building the best products and systems on Earth (and the universe!).

Monitoring and observability provide key components in the transparency of the DevOps culture. If you want to move from a paradigm where the security team is solely responsible for security to one where security is everyone's responsibility, you must give everyone visibility into the data about security which is so critical to making good decisions about security. While you must be careful about sharing confidential information, sharing information about vulnerabilities in a transparent way across all technology teams can be a great way to drive improvement in those areas.

Automation helps free engineers from manual tasks, enabling them to focus on higher-level engineering problems. SIEM and SOAR can significantly reduce operational overhead, thus automating the most common activities in security engineering. In addition, this automation can help reduce the noise from monitoring telemetry to help focus on the important signals that indicate valid security threats.

Through technology, you enable the culture of DevOps. Automation can help drive greater flow of value to your customers, as can your CI/CD pipelines. By building security automation, you can ensure that security is done as part of everything your organization does instead of being added on as an afterthought.